
Konzeption und Implementierung eines Webservice zum Scheduling der SYNCROTESS BM Transportoptimierung

im Studiengang Angewandte Mathematik und Informatik

von Niklas Paar

Matrikelnummer: 3637975

Diese Arbeit wurde betreut von

1. Prüfer: Prof. Dr. rer. nat. Alexander Voß
2. Prüfer: Frank Fleischhack

Aachen, Dezember 2025

Sperrvermerk

Die vorliegende Seminararbeit enthält vertrauliche Informationen der INFORM GmbH. Sie darf ausschließlich von den Prüfern und den berechtigten Personen der Fachhochschule Aachen eingesehen werden. Eine Weitergabe an Dritte sowie eine Veröffentlichung, Vervielfältigung oder Verwertung der Inhalte ist ohne ausdrückliche Genehmigung der INFORM GmbH nicht gestattet.

Inhaltsverzeichnis

1. Einleitung	6
1.1. Hintergrund und Motivation	6
1.2. Ziel	7
1.3. Vorgehensweise	7
2. Ist-Analyse	9
2.1. SYNCROTES BM und der Tserv	9
2.2. Optimierer	10
2.2.1. coflo	10
2.2.2. ftlopt und MIXOR	11
2.3. Aktueller Ablauf einer Optimierung	12
2.3.1. Dispatch-Gruppen und Optimierungstypen	12
2.3.2. Bestehendes Scheduling: Mehrere FIFO-Warteschlangen	13
2.4. Probleme und Einschränkungen des Ist-Zustands	14
2.4.1. Starre, hartcodierte Parallelitätsstruktur	14
2.4.2. Kein typenübergreifendes Scheduling und fehlende Priorisierung	14
2.4.3. Fork-Mechanismus	15
2.4.4. Ausschließliche Erreichbarkeit über den Tserv	15
2.4.5. Fehlende Ressourcenkontrolle und Laufzeitüberwachung	15
2.4.6. Zusammenfassung	16
3. Anforderungen an den Scheduler-Webservice	17
3.1. Vorbedingungen	17
3.2. Anwendungsfälle	18
3.2.1. Optimierungsauftrag einreichen mit leerer Warteschlange, kein Optimierer läuft	18
3.2.2. Optimierungsauftrag einreichen mit leerer Warteschlange, ein Optimierer läuft	18
3.2.3. Optimierungsauftrag einreichen mit gefüllter Warteschlange, maximale Optimierer laufen	18
3.2.4. Optimierungsauftrag einreichen mit voller Warteschlange, maximale Optimierer laufen	19

3.2.5.	Optimierungsauftrag in Warteschlange, Optimierer wird fertig	19
3.2.6.	Eingabedaten unvollständig	19
3.2.7.	Arbeitsspeicher wird überlastet	19
3.2.8.	Optimierung dauert zu lange	20
3.2.9.	Scheduler wird ohne oder mit falscher Authentifizierung angesprochen	20
3.3.	Softwarearchitektur und Technologien	20
3.4.	Beschreibung der Algorithmen	21
3.4.1.	Non-Preemptive Priority Scheduling	21
3.4.2.	Preemptive Priority Scheduling	21
3.4.3.	First in First Out (FIFO)	22
3.4.4.	Deadline Monotonic Scheduling (DMS)	22
3.4.5.	Least Slack Time First (LSF)	23
3.4.6.	Highest Response Ratio Next	23
3.4.7.	Shortest Job First (SJF)	24
3.4.8.	Shortest Remaining Time First	24
3.5.	Selektierung der Algorithmen	25
3.5.1.	Ausschlusskriterien	25
3.5.2.	Ausschluss ungeeigneter Algorithmen	26
3.6.	Zusammenfassung	27
4.	Konzeptionierung	28
4.1.	Datenstrukturen	28
4.1.1.	Eingabe-Datentypen	28
4.1.2.	Ausgabe-Datentypen	28
4.2.	Aufbau des Webservice	29
4.2.1.	Überblick	29
4.2.2.	Ablauf	29
4.2.3.	Systemarchitektur	30
4.2.4.	Technologie-Stack	30
4.3.	Beschreibung der ausgewählten Algorithmen	31
4.3.1.	First In First Out (FIFO)	31
4.3.2.	Deadline Monotonic Scheduling (DMS)	32
4.3.3.	Vergleich der Algorithmen	34
4.4.	Analyse der Algorithmen	34
4.4.1.	Implementierung der Prototypen	34
4.4.2.	Messgrößen	35
4.4.3.	Versuchsaufbau	35
5.	Evaluation	36

6. Fazit & Ausblick	37
-------------------------------	----

1. Einleitung

Diese Bachelorarbeit thematisiert die Implementierung und Konzeptionierung eines Webservice zum Scheduling (OptimSchedulers) der SYNCROTESS BM Transportoptimierung. Diese Arbeit wird von der INFORM GmbH (Institut für Operations Research und Management GmbH) betreut, einem Softwareunternehmen, das auf Operations Research spezialisiert ist.

1.1. Hintergrund und Motivation

Die von INFORM entwickelte Softwarelösung SYNCROTESS BM richtet sich an Unternehmen aus der Baustoff- bzw. der Logistikbranche und deckt unter anderem die Planung von Transporten ab. Ein Bestandteil dieser Lösung ist die Transportoptimierung, wodurch Dispositionsentscheidungen sinnvoll getroffen werden können. Logistische Abläufe sollen dadurch effizienter und schneller optimiert werden. Damit diese Optimierungsläufe durchgeführt werden können, werden spezialisierte Programme - sogenannte Optimierer - eingesetzt.

Derzeit werden die Optimierer innerhalb des *Tserv* ausgeführt. Dies ist das Backend des SYNCROTESS-Produkts für das Transportmanagement. Der *Tserv* enthält die Geschäftslogik und führt die einzelnen Optimierer intern aus. Die Optimierer-Prozesse werden momentan als Fork, also als neuer Prozess auf demselben Host-System, ausgeführt. Dadurch sind die Prozesse durch die Hardware des Host Systems limitiert. Diese kann nur eingeschränkt oder umständlich angepasst werden. Zusammengefasst führt das zu folgenden Einschränkungen:

- Die Optimierungsprozesse werden nur bedingt parallel ausgeführt
- Die Optimierer sind ausschließlich über den *Tserv* ansprechbar
- Geringe Skalierbarkeit
- Keine Priorisierung der Optimierungsprozesse

Insbesondere kann die fehlende Priorisierung und die fehlende Parallelisierung der einzelnen Optimierungsprozesse bei zeitkritischen Szenarien zu Problemen führen, wenn die Optimierung durch weniger relevante Prozesse blockiert wird. Es fehlt daher ein Mechanismus, der die Prozesse priorisiert und parallelisiert abarbeiten kann.

1.2. Ziel

Das Ziel dieser Arbeit ist es, einen Webservice zu konzipieren und zu implementieren. Der Webservice soll die Ausführung der Optimierungsprozesse vom Tserv entkoppeln und über eine REST-Schnittstelle steuern. Die Optimierung soll also in einem eigenen Prozess und nicht zwingend in dem Tserv laufen. Dabei sollen mehrere Optimierungsprozesse parallel laufen können. Eingehende Anfragen sollen über eine Warteschlange koordiniert werden; Grenzen wie maximale Laufzeit und maximale Arbeitsspeicherauslastung sollen dabei berücksichtigt werden.

Um die Reihenfolge der abzuarbeitenden Prozesse festlegen zu können, soll eine geeignete Scheduling Strategie gefunden werden. Dazu müssen einzelne Algorithmen untersucht und verglichen werden. Eine Herausforderung dabei ist, dass die Ausführungszeiten der Optimierer im Voraus unbekannt sind und auch nicht ausreichend abgeschätzt werden können. Durch eine Anforderungsanalyse werden verschiedene Scheduling-Verfahren vorgestellt, bewertet und anhand von definierten Kriterien evaluiert.

Die Implementierung einzelner Algorithmen dient zunächst ausschließlich der Evaluierung und ist nicht für den Produktiveinsatz vorgesehen. Stattdessen ist das Ziel, eine fundierte Entscheidungsgrundlage für die Auswahl einer geeigneten Scheduling-Strategie zu schaffen.

1.3. Vorgehensweise

Zunächst soll eine Ist-Analyse des bestehenden Systems durchgeführt werden. Dazu wird der Aufbau des Tservs und speziell die Optimierungsverfahren erläutert.

Darauf aufbauend werden Anforderungen definiert, die der neue Webservice erfüllen muss. Hierbei werden funktionale Anforderungen mithilfe von definierten Anwendungsfällen erarbeitet und technische Randbedingungen festgelegt.

Im Anschluss folgt eine Recherche verschiedener Scheduling-Algorithmen, die hinsichtlich ihrer Eignung bewertet werden. Anschließend werden die Algorithmen, die gegen Ausschlusskriterien verstoßen, verworfen. Die verbleibenden Verfahren werden weiter auf erarbeitete Kriterien untersucht.

Abschließend wird auf der Basis der Untersuchungen ein Scheduling-Algorithmus für die Implementierung des Webservices festgelegt.

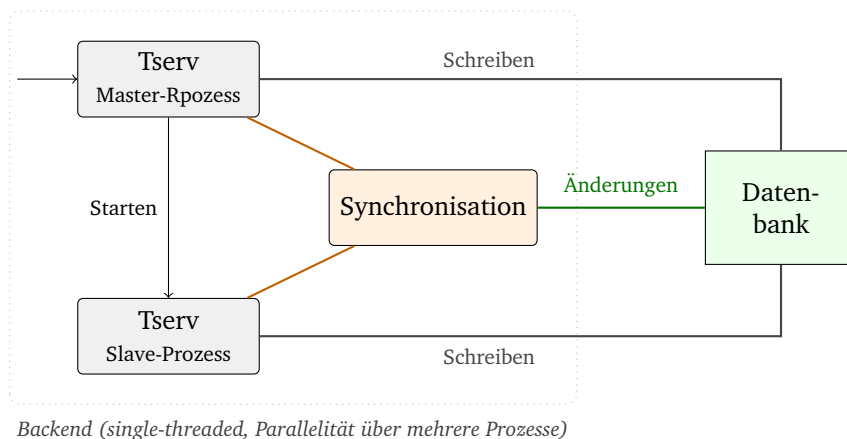
2. Ist-Analyse

Um zu verstehen, warum die Implementierung eines Scheduling-Webservices sinnvoll ist, muss man zunächst verstehen, wie das bestehende System arbeitet. Dieses Kapitel beschreibt nun den Aufbau der aktuellen Implementierung des Tserv, dem zentralen Anwendungsserver der SYNCROTESS-BM-Plattform. Zusätzlich wird der darin beinhaltete Ablauf der Optimierung beschrieben. Abschließend werden die Einschränkungen und Probleme des Ist-Zustands herausgearbeitet, die den Ausgangspunkt und die Voraussetzung für die Konzeption des OptimSchedulers bilden.

2.1. SYNCROTESS BM und der Tserv

SYNCROTESS BM ist eine von der INFORM GmbH entwickelte Softwarelösung für die Transportplanung in der Logistikbranche, speziell im Bereich der Baustoffe. Kern dieser Plattform ist der Tserv, ein zentraler Anwendungsserver, welcher die Geschäftslogik der Software beinhaltet, bündelt und die Kommunikation zwischen den verschiedenen Systemkomponenten koordiniert.

Dieser Tserv ist eine komplexe und über Jahre gewachsene Serveranwendung, die mit dem Betriebssystem Linux betrieben wird. Seine Aufgabe ist es, Stammdaten und Datenabfragen zu verarbeiten. Darüber hinaus verarbeitet er CRUD-Operationen zu den einzelnen Feldern. Zusätzlich steuert er den gesamten Lebenszyklus der Transportvorgänge. Ein wesentlicher Bestandteil ist dabei die Transportoptimierung. Auf Anfrage führt der Tserv spezialisierte Optimierungsalgorithmen aus, welche auf Grundlage der vorgelegten Daten optimale Dispositionsentscheidungen bestimmen. Zu den Daten, welche übermittelt werden zählen zum Beispiel Auftragsdaten (Mengen, Zeitfenster, etc.), Öffnungszeiten, Standorte, Fahrzeugtypen.



Der Tserv Master-Prozess wird dabei von außen über eine Mutation angesprochen. Dieser kann dabei durch einen Fork einzelne slave-Prozesse starten. Wenn er von einer Mutation angesprochen wird, schreibt er die Änderung in eine Datenbank und bei einer Datenbankänderung wird der Stand von allen slave-Prozessen angepasst. Der Tserv kann dabei von außen von einem Frontend, einem ERP-System oder einem Telematik-System angesprochen werden.

2.2. Optimierer

Die Optimierungslogik ist nicht im Tserv selbst implementiert, sondern in eigenständigen, separaten Programmen - den sogenannten Optimierern. Die Optimierer werden von Tserv zur Laufzeit aufgerufen und berechnen auf Basis einer JSON-Eingabedatei auch eine JSON-Ausgabedatei. Die Optimierer sind dabei in C++ implementiert. Zusätzlich werden externe Bibliotheken, wie *Boost* genutzt und die Optimierer werden mit *Makefile* gebaut. Diese Skripte sind dann als standalone lauffähig, werden aber ausschließlich durch den Tserv gestartet.

2.2.1. coflo

coflo ist ein weiterer Optimierer, der speziell für die Zementoptimierung im Bereich Full Truck Load (FTL) entwickelt wurde. Full Truck Load bezeichnet Transporte, wobei ein einzelner Auftrag das gesamte Fahrzeug auslastet - im Gegensatz zu Teilladungen, wobei mehrere Aufträge auf einem Fahrzeug gebündelt werden. *coflo* adressiert damit den Transport von einem Massengut, wie beispielsweise Zement, während *ftlopt* auf Beton optimiert ist.

Analog zu *ftlopt* wird auch *coflo* vom Tserv über ein Shell-Skript-Template gestartet und kommuniziert dateibasiert. im Gegensatz zu *ftlopt* nutzt *coflo* als Ein- und Ausgabeformat CSV. Die Herausforderung, die bei der Zementoptimierung existiert ist, dass bei Zement keine Lieferkette existiert und somit eine Abhängigkeit mehrerer Lieferungen voneinander entsteht. Diese Probleme werden durch *coflo* adressiert.

```
coflo inst
```

In dem Kommando sieht man, dass der *coflo* Optimierer lediglich eine Eingabedatei erhält. Ein Arbeitsverzeichnis und die Benennung der Ausgabedatei sind fest definiert.

2.2.2. *ftlopt* und MIXOR

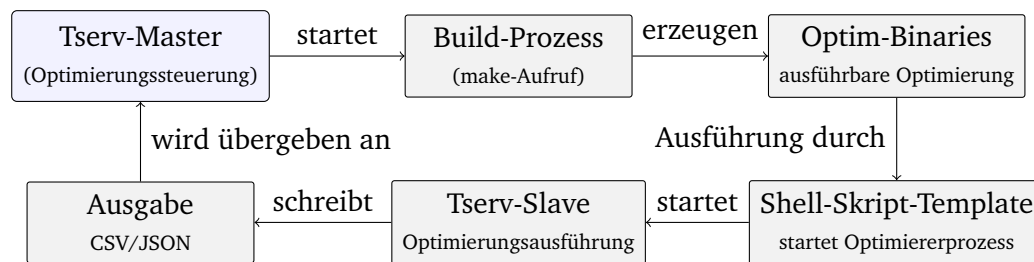
ftlopt ist das zentrale Optimierungsframework für die Transportoptimierung. Es optimiert die Zuordnung von Kundenaufträgen zu LKW-Schichten und Werken. Die Basis dafür sind verschiedenste Nebenbedingungen und Zielfunktionen. Ein konkreter Optimierungsansatz für den Transportbeton-Geschäftskontext, den *ftlopt* zur Verfügung stellt, ist der Algorithmus Ready Mix Concrete Operations Research (MIXOR).

MIXOR unterstützt einen Mehrschicht-Betrieb, also sowohl Tag- als auch Nachtschichten; somit ist die Optimierung auch flexibel und über 0 Uhr hinaus möglich. Darüber hinaus unterstützt MIXOR eine dynamische Pausenplanung anhand eines vorgegebenen Musters. Die Ein- und Ausgabe ist jeweils als JSON realisiert. Ein typischer Aufruf sieht wie folgt aus:

```
ftlopt -i ftlopt1.json -w /tmp/workdir/ -s ftlopt1.sol.json
```

In dem Kommando sieht man, dass über *-u* die Eingabedatei übergeben wird. Durch *-w* wird das Arbeitsverzeichnis übergeben, und mit *-s* wird die Ausgabedatei mit dem Optimierungsergebnis definiert.

2.3. Aktueller Ablauf einer Optimierung



Die grundlegenden Aufgaben der einzelnen Optimierer sind nun bekannt. Nun wird der grundlegende Ablauf einer Optimierung erläutert. Momentan läuft die Optimierung im System wie folgt ab:

- Der Tserv enthält die Optimierungssteuerung als internes Modul
- Wenn der Build-Prozess des Tservs läuft, dann werden die Optimierer-Binaries durch den *make*-Aufrufs erzeugt und unter `$INSTROOT/bin/` abgelegt
- Dort wird dann das passende Shell-Skript-Template instanziiert und ausgeführt, welches den entsprechenden Optimierer-Prozess startet.

Der Optimierer-Prozess wird dabei als *Fork* gestartet - das bedeutet, dass ein slave-Prozess auf demselben Host-System erzeugt wird, der die Ausführung des Optimierers übernimmt. Dabei wird eine Dateischnittstelle zwischen Tserv und Optimierer verwendet. Der Tserv schreibt also die Eingabedatei im CSV- oder JSON-Format, die der Tserv-Master einliest und weiterverarbeitet.

2.3.1. Dispatch-Gruppen und Optimierungstypen

Eine Dispatch Gruppe repräsentiert eine logische Einheit der Transportplanung in der Software der INFORM GmbH. Diese stellt eine Planungsregion aus ein oder mehreren Werken dar. für jede Dispatchgruppe wird unabhängig optimiert, und innerhalb jeder Dispatchgruppe gibt es zusätzlich folgende Optimierungstypen:

- **RT (Realtime):** Dieser Optimierungstyp beschreibt die Echtzeioptimierung unserer Software. Hier wird auf den direkten Fahrzeugstatus oder Änderungen in der Schichtplanung reagiert, und anhand dessen kann geplant werden.

- **OEVO, OEV1, OEV2:** Dieser Optimierungstyp beschreibt die Varianten der *Optimierten Endvorschau*. Planungsläufe, die den Tagesplan auf Basis aller vorliegenden Aufträge und Ressourcen neu berechnen und dem Disponenten als Entscheidungsgrundlage dienen.
- **PP (Preplanning):** Dieser Optimierungstyp dient vor allem der Pflege der Fahrzeugflotte für den Folgetag (Menge, Startzeiten) und des Auftragsbuches. Zudem gibt es diesen Optimierungstyp nur in coflo Optimierung.

Die Optimierungstypen sind bei allen Dispatchgruppen identisch. Darüber hinaus wird für jeden dieser Optimierungstypen ein eigener Tserv Fork-Prozess erstellt, sodass RT, OEVO und PP innerhalb einer Dispatch-Gruppe parallel ausgeführt werden können. Oev1 und Oev2 bilden dabei eine kleine Ausnahme, da diese auf demselben Fork-Prozess laufen und dabei sich immer alternierend mit der Ausführung abwechseln.

2.3.2. **Bestehendes Scheduling: Mehrere FIFO-Warteschlangen**

Pro Optimierungstyp existiert also ein eigener Fork Prozess. Somit existiert, dann auch eine eigene Warteschlange für jeden dieser Typen. Diese wird nach dem *First-In-First-Out-Prinzip* (FIFO) abgearbeitet. Die Anfrage, die zuerst eintrifft wird also zuerst abgearbeitet. In der Praxis tritt es allerdings eigentlich nicht auf, dass die Warteschlange verwendet werden muss, da man in der Software nicht mehrere RT,PP oder OEV Optimierungen anstoßen kann. Ausnahme bilden hierbei OEV1 und OEV2. Bei diesen Prozessen muss der eine auf den anderen warten, wobei alternierend gewechselt wird.

Das bedeutet, dass das bestehende System bereits über eine rudimentäre Form von Parallelität und Scheduling verfügt. Diese ist jedoch starr, da sie ausschließlich auf der festen Struktur der Dispatch-Gruppen und Optimierungstypen basiert. Zudem muss das bestehende System über eine entsprechende Anzahl an Kernen verfügen, um gegenseitig Blockierungen zu vermeiden. Die Anzahl der Kerne hängt von der Anzahl der Distpatch-Gruppen und OEV-Tagen ab.

2.4. Probleme und Einschränkungen des Ist-Zustands

Der beschriebene Ablauf des Optimierungsablaufs ist funktional und verfügt bereits über eine rudimentäre Form von Scheduling und Parallelität. Das Problem ist, dass der Ablauf dennoch strukturelle Einschränkungen aufweist, die mit steigenden Anforderungen an die Plattform problematisch werden können. So erfordert etwa die Anbindung eines neuen Optimierers oder die Einführung eines zusätzlichen Optimierungstyps stets Änderungen am Tserv-Quellcode selbst. Ebenso wächst mit steigender Anzahl von Dispatch-Gruppen und gleichzeitigen Optimierungsanfragen der Druck auf die Hardware des Host-Systems, da alle Prozesse denselben Rechner teilen. Auf der anderen Seite steigen auch die Anforderungen an die externe Nutzbarkeit der Optimierer: Szenarien, in denen externe Systeme oder Services die Optimierung direkt anstoßen sollen, lassen sich mit der bestehenden Architektur nicht abbilden. Die folgenden Abschnitte beschreiben die einzelnen Einschränkungen im Detail.

2.4.1. Starre, hartcodierte Parallelitätsstruktur

Die Parallelität im derzeitigen System ist fest an die Struktur der Dispatch-Gruppen und der verschiedenen Optimierungstypen (RT, OEVO, ...) gebunden. Es läuft für jeden Typen ein Prozess parallel, dies passt sich nicht an die Anforderungen an. Eine Anpassung dieser Struktur, beispielsweise um zusätzliche Parallelität unabhängig von Dispatchgruppen oder Optimierungstypen würde die Struktur auflockern und das hinzufügen neuer Optimierungstypen vereinfachen. Dadurch würde auch die Parallelität eine Eigenschaft der Systemarchitektur werden und kein konfiguriertes Verhalten sein.

2.4.2. Kein typenübergreifendes Scheduling und fehlende Priorisierung

Jede Warteschlange ist von der jeweils anderen vollständig isoliert. Zudem werden die Warteschlangen strikt nach FIFO abgearbeitet. Eine Priorisierung über eine Typgrenze hinweg ist daher unmöglich. Beispiel: Eine Priorisierung über Typgrenzen hinweg ist nicht möglich: Eine dringende RT-Optimierung kann nicht vorgezogen werden, wenn in ihrer Warteschlange bereits ein anderer RT-Lauf

wartet – und sie kann erst recht nicht Ressourcen beanspruchen, die gerade ein OEV- oder PP-Prozess belegt.

Umgekehrt kann ein weniger wichtiger Optimierungstyp keine Ressourcen für einen wichtigeren Typen freigeben. Somit können weniger wichtigere PP-Prozesse auch wichtigere RT-Prozesse überholen.

Durch das Fehlen einer übergeordneten Scheduling-Logik kann es in zeitkritischen Szenarien zu unnötigen Verzögerungen kommen.

2.4.3. Fork-Mechanismus

Dadurch, dass alle Optimierer-Prozesse als Fork auf demselben Host-System gestartet werden, teilen sie sich auch die Ressourcen des Systems, also auch Arbeitsspeicher und CPU-Kerne. Die Rechenkapazität ist damit direkt an die Hardware des Host-Systems gebunden, die sich nur eingeschränkt oder mit hohem Aufwand anpassen lässt. Eine flexiblere Skalierung wäre also wünschenswert.

Ein weiteres Problem welches durch den Fork-Mechanismus entsteht ist, dass immer ein gesamter Tserv für eigentlich nur einen kleinen Teil des Tservs geforkt wird. Dadurch wird zusätzlich unnötig Arbeitsspeicher verbraucht.

2.4.4. Ausschließliche Erreichbarkeit über den Tserv

Die Optimierung ist ausschließlich über den Tserv ansprechbar, obwohl die Programme bzw. die Skripte technisch gesehen bereits jetzt als standalone, also unabhängig, lauffähig sind. Dennoch sind sie im derzeitigen System in den Tserv eingebunden. eine unabhängige Nutzung, beispielsweise für eine externe Anbindung oder den Betrieb in anderer Infrastruktur ist nicht möglich ohne auch den Tserv zu nutzen.

2.4.5. Fehlende Ressourcenkontrolle und Laufzeitüberwachung

Derzeit existiert keine Überwachung der Systemressourcen bei der Optimierungsausführung. Bei fehlerhaften oder lang laufenden Optimierungen gibt es keinen automatischen Abbruchmechanismus. Zudem wird vor dem start eines Optimierers nicht geprüft, ob genug Arbeitsspeicher für weitere Prozesse vorhanden ist. Dies kann bei hoher Auslastung zu Instabilität des Gesamtsystem führen.

2.4.6. Zusammenfassung

Tabelle 2.1 fasst die identifizierten Einschränkungen des Ist-Zustands zusammen und stellt ihnen die jeweils angestrebte Verbesserung durch den OptimScheduler gegenüber.

Einschränkung (Ist)	Ziel (Soll)
Parallelität starr an Dispatch-Gruppen und Optimierungstypen gebunden; nicht konfigurierbar	Flexibel konfigurierbare Parallelität, unabhängig von fixer Typstruktur
Warteschlangen isoliert und strikt FI-FO; kein typenübergreifendes Scheduling – weniger wichtige PP-Prozesse können RT-Prozesse überholen	Priorisierungsfähiges Scheduling über alle Optimierungstypen hinweg
Fork-Mechanismus: Rechenkapazität an Host-Hardware gebunden; Fork des gesamten Tserv verursacht unnötigen Speicherverbrauch	Entkopplung vom Tserv; flexible Skalierbarkeit und gezielter Ressourceneinsatz
Optimierer ausschließlich über den Tserv erreichbar, obwohl technisch standalone lauffähig	Eigenständiger Webservice
Keine Ressourcenüberwachung; kein automatischer Abbruch bei fehlerhaften Optimierungen	Timeout-Handling und Überwachung der Systemauslastung

Tabelle 2.1.: Einschränkungen des Ist-Zustands und angestrebte Verbesserungen

Diese Defizite bilden die fachliche Grundlage für die Anforderungsanalyse im folgenden Kapitel.

3. Anforderungen an den Scheduler-Webservice

Da nun die Probleme des Ist-Zustandes des Tserv-Systems identifiziert wurden, werden in diesem Kapitel nun die Anforderungen an den zu entwickelnden OptimScheduler definiert. Diese Anforderungen setzen sich aus funktionalen Anforderungen, die das erwartete Verhalten des Systems beschreiben, sowie technische, also nicht funktionale, Anforderungen, die die Umsetzung und Integration in die bestehende Infrastruktur aufzeigen und dabei Grenzen für die Scheduling-Algorithmen setzen. Desweiteren werden die verschiedenen Scheduling Algorithmen vorgestellt und anschließend wird eine Vorselektion dieser stattfinden.

3.1. Vorbedingungen

Der OptimScheduler soll in einem speziellen Umfeld entwickelt und betrieben werden. Die folgenden Vorbedingungen definieren die Rahmenbedingungen:

- Der OptimScheduler wird als Webservice in Java entwickelt. Dabei wird das Framework Spring Boot verwendet
- Angesprochen wird der Scheduler über eine REST-Schnittstelle
- Der OptimScheduler wird als alleinstehender Dienst implementiert, ohne direkte Verbindung mit dem Tserv
- Er kann auf einem separaten System gehostet werden, muss aber mit dem Tserv kommunizieren können
- Der Scheduler muss mit den Ein- und Ausgabeformaten JSON und CSV kompatibel sein
- Der OptimScheduler soll mehrere Optimierungen parallel bearbeiten können
- Die Schnittstelle muss auch eine Authentifizierung abfragen

3.2. Anwendungsfälle

Im nachfolgenden wird häufig von Warteschlangen gesprochen. Damit ist nicht eine klassische FIFO-Warteschlange gemeint, sondern eine Warteschlange, die nach dem später festgelegten Scheduling-Algorithmus abgearbeitet wird.

3.2.1. Optimierungsauftrag einreichen mit leerer Warteschlange, kein Optimierer läuft

Ein externer Dienst (z.B. der Tserv) versendet einen Optimierungsauftrag an den Scheduler. Dieser Auftrag enthält den Payload. Dieser besteht unter anderem aus der Eingabedatei (JSON/CSV) und der Auswahl des Optimierers. Der Scheduler prüft die Daten und die Optimiererauswahl. Da die Warteschlange leer ist und kein Optimierer läuft, kann der Auftrag direkt bearbeitet und anschließend die Ausgabe zurück an den externen Dienst gesendet werden.

3.2.2. Optimierungsauftrag einreichen mit leerer Warteschlange, ein Optimierer läuft

Wie auch bei dem vorherigen Anwendungsfall kommt der Auftrag von einem externen Dienst. Ein Optimierer läuft im Moment. Da mehrere parallel laufen können, wird der Auftrag parallel abgearbeitet und die Ausgabe wird anschließend zurück an den Absender geschickt.

3.2.3. Optimierungsauftrag einreichen mit gefüllter Warteschlange, maximale Optimierer laufen

Wie auch bei den vorherigen Anwendungsfällen kommt hier ein Auftrag von einem externen Dienst an. Anschließend stellt der Scheduler fest, dass die maximale Anzahl an Optimierern läuft und die Warteschlange gefüllt ist. Anschließend wird der Auftrag in die Warteschlange hinzugefügt. Die Warteschlange wird anschließend nach dem festgelegten Algorithmus abgearbeitet, und irgendwann kommt der Auftrag dran; anschließend wird das Ergebnis an den Absender geschickt.

3.2.4. Optimierungsauftrag einreichen mit voller Warteschlange, maximale Optimierer laufen

Wie auch bei den vorherigen Anwendungsfällen kommt hier ein Auftrag von einem externen Dienst an. Anschließend stellt der Scheduler fest, dass die maximale Anzahl an Aufträgen in der Warteschlange bereits erreicht ist. Deshalb wird hier eine Fehlermeldung an den Absender gesendet, die beinhaltet, dass die maximale Auslastung des OptimSchedulers erreicht sei.

3.2.5. Optimierungsauftrag in Warteschlange, Optimierer wird fertig

Wenn man eine Warteschlange hat, wird diese irgendwann abgearbeitet. Wenn also ein Auftrag in der Warteschlange ist, wird dieser begonnen, sobald ein Auftrag abgearbeitet wurde und somit ein Platz frei wird.

3.2.6. Eingabedaten unvollständig

Wenn ein Auftrag im Scheduler ankommt, der unvollständige oder nicht zusammenpassende Daten beinhaltet, wird eine passende Fehlermeldung an den Absender zurückgesendet. Darunter zählt zum Beispiel ein ungültiges Eingabeformat oder eine leere Eingabedatei. Ein weiterer Grund für eine solche Fehlermeldung könnte die fehlende Auswahl an Optimierern sein.

3.2.7. Arbeitsspeicher wird überlastet

Es kommt erneut ein Auftrag von einem externen Dienst an. Nun ist die Warteschlange leer und ein Platz bei den bereits laufenden Prozessen frei. Das Problem ist nun, dass die Auslastung des Arbeitsspeichers allerdings schon ans Maximum geht. In diesem Fall soll der Optimierungsauftrag warten, bis die Auslastung wieder in einem unkritischen Bereich ist.

3.2.8. Optimierung dauert zu lange

In dem Payload, der von dem externen Dienst an den Scheduler geschickt wird, steht auch eine maximale Dauer. Wenn diese erreicht ist, soll der Prozess abgebrochen werden und eine Fehlermeldung an den Absender gesendet werden.

3.2.9. Scheduler wird ohne oder mit falscher Authentifizierung angesprochen

Ein Auftrag wird von einem externen Dienst an den OptimScheduler geschickt. Dieser enthält keine Authentifizierung oder eine falsche Authentifizierung. In diesem Fall wird sofort eine Fehlermeldung an den Absender geschickt, dass eine Authentifizierung notwendig ist.

3.3. Softwarearchitektur und Technologien

- **REST-API:** Der Scheduler stellt für die Optimierungsabfragen eine REST-Schnittstelle zur Verfügung
- **Eingabeformat:** Ein- und Ausgabe wird im Format JSON und CSV akzeptiert und verarbeitet
- **Parallelisierung:** Der Scheduler soll mehrere Prozesse für die Optimierung parallel verarbeiten können
- **Scheduling-Algorithmus:** Die Reihenfolge der Abarbeitung der wartenden Prozesse, wird durch einen Scheduling-Algorithmus bestimmt
- **Ressourcenüberwachung:** Der Webservice muss den verfügbaren Arbeitsspeicher und die CPU-Auslastung vor dem Start eines neuen Prozesses prüfen
- **Laufzeitkontrolle:** Der Scheduling-Algorithmus muss die Laufzeitgrenzen der Optimierungsprozesse beachten

3.4. Beschreibung der Algorithmen

Im folgenden werden einige Varianten von Scheduling Algorithmen vorgestellt.

3.4.1. Non-Preemptive Priority Scheduling

Funktionsweise: Jeder Auftrag bekommt eine statische Priorität zugewiesen. Die Aufträge mit der höheren Priorität werden gegenüber den mit geringerer Priorität bevorzugt verarbeitet. Ein laufender Prozess kann nicht unterbrochen werden auch wenn ein neuer Prozess mit höherer Priorität dazu kommt. Der neue Auftrag wird erst gestartet, nach dem ein vorheriger fertig geworden ist.

Eigenschaften:

- + Relativ einfach zu implementieren
- + Ermöglicht es, zwischen wichtigeren und weniger wichtigen Aufträgen zu differenzieren
- Kann bei hoher Auslastung zu *Starvation* von Aufträgen mit geringer Priorität führen
- Verbleibende Bearbeitungsdauer erforderlich?: Nein
- Scheduling-Typ: nicht präemptiv

3.4.2. Preemptive Priority Scheduling

Funktionsweise: Auch hier bekommt jeder Auftrag eine statische Priorität hinzugefügt. Im Vergleich zum *Non-Preemptive Priority Scheduling* kann hier eine höhere Priorität einen laufenden Prozess unterbrechen und ersetzen.

Eigenschaften:

- + Ermöglicht es, zwischen wichtigeren und weniger wichtigen Aufträgen zu differenzieren
- + Schnellere Reaktion bei wichtigen Aufträgen
- Komplexere Implementierung
- Abgebrochene Prozesse müssen gespeichert und später fortgesetzt werden

- Kann bei hoher Auslastung zu *Starvation* von Aufträgen mit geringer Priorität führen
- Verbleibende Bearbeitungsdauer erforderlich?: Nein
- Scheduling-Typ: präemptiv

3.4.3. First in First Out (FIFO)

Funktionsweise: Aufträge werden in der Reihenfolge bearbeitet, in der sie ankommen. Der erste eingegangene Auftrag wird auch als erstes bearbeitet. Ein neuer Auftrag wird nur dann gestartet, wenn der vorherige abgeschlossen wurde.

Eigenschaften:

- + Einfach zu implementieren
- Keine Priorisierung möglich
- Risiko: *Convoy Effect* kann auftreten. Das heißt, dass ein langer Prozess alle anderen blockiert
- Verbleibende Bearbeitungsdauer erforderlich?: Nein
- Scheduling-Typ: nicht präemptiv

3.4.4. Deadline Monotonic Scheduling (DMS)

Funktionsweise: DMS ist, wie das *Non-Preemptive Priority Scheduling* ein statisches Prioritäts-Scheduling-Verfahren. Hier wird die Deadline also auch statisch bestimmt, aber nicht direkt gesetzt, sondern mit einer Deadline also einer maximalen Bearbeitungszeit verknüpft. Hier gilt also: Der Auftrag mit der kürzeren Zeit zwischen jetzt und der Deadline wird als nächstes genommen.

Eigenschaften:

- + Ermöglicht es, zwischen wichtigeren und weniger wichtigen Aufträgen zu differenzieren
- + Für Echtzeitsysteme mit bekannten Deadlines geeignet
- + Garantiert die Einhaltung von Deadlines, sofern die Rechenleistung ausreichend ist

- Jeder Auftrag muss ein Zeitlimit oder eine Deadline halten, um die Priorität zu bestimmen
- Verbleibende Bearbeitungsdauer erforderlich?: Nein
- Scheduling-Typ: nicht präemptiv

3.4.5. Least Slack Time First (LSF)

Funktionsweise: LSF berechnet für jeden Auftrag die *Slack Time* (Slack Time = Deadline - Restlaufzeit - aktuelle Zeit). Dabei hat immer der Auftrag mit der geringsten *Slack Time* die höchste Priorität. Dieser Algorithmus berücksichtigt also zusätzlich die verbleibende Rechenzeit.

Eigenschaften:

- + Ermöglicht es, zwischen wichtigeren und weniger wichtigen Aufträgen zu differenzieren
- + Dynamische Berechnung der Priorität
- + Berücksichtigt Bearbeitungsdauer und Deadline
- + Minimiert die Anzahl verpasster Deadlines
 - Erfordert genaue Schätzung der Restlaufzeit
- Verbleibende Bearbeitungsdauer erforderlich?: Ja
- Scheduling-Typ: präemptiv

3.4.6. Highest Response Ratio Next

Funktionsweise: Bei diesem Scheduling-Algorithmus werden die Aufträge nach der sogenannten *Response Ration* sortiert. Die Response Ratio wird berechnet als:

$$\frac{\text{Wartezeit} + \text{Bearbeitungszeit}}{\text{Bearbeitungszeit}}.$$

Eigenschaften:

- + Ermöglicht es, zwischen wichtigeren und weniger wichtigen Aufträgen zu differenzieren
- + Berücksichtigt die Bearbeitungszeit und die Bearbeitungsdauer
- + Reduziert *Starvation*

- Erfordert möglichst genaue Schätzung der Bearbeitungsdauer
- Verbleibende Bearbeitungsdauer erforderlich?: Ja
- Scheduling-Typ: nicht präemptiv

3.4.7. Shortest Job First (SJF)

Funktionsweise: Hier wird immer der Auftrag als nächstes gewählt, der die geringste Bearbeitungszeit hat. Der Prozess läuft, wenn er gestartet wurde, bis zum Ende durch. Ziel dieses Algorithmus ist es, die Wartezeit zu minimieren, indem die kürzesten Aufträge immer zuerst abgearbeitet werden.

Eigenschaften:

- + Ermöglicht es, zwischen wichtigeren und weniger wichtigen Aufträgen zu differenzieren
- + Minimiert die durchschnittliche Wartezeit
 - Benötigt eine relativ genaue Schätzung der Bearbeitungszeiten
 - Risiko von *Starvation* bei Aufträgen mit hoher Bearbeitungszeit
- Verbleibende Bearbeitungsdauer erforderlich?: Ja
- Scheduling-Typ: nicht präemptiv

3.4.8. Shortest Remaining Time First

Funktionsweise: Dieser Algorithmus ist die präemptive Variante von SJF. Hier wird der Prozess mit der geringsten geschätzten Restlaufzeit bevorzugt. Wenn ein neuer Auftrag also ankommt und dessen Restlaufzeit geringer ist, als die des gerade laufenden Prozesses, dann wird der aktuell laufende Prozess unterbrochen und der neue Auftrag dafür abgearbeitet.

Eigenschaften:

- + Ermöglicht es, zwischen wichtigeren und weniger wichtigen Aufträgen zu differenzieren
- + Minimiert die durchschnittliche Wartezeit
 - Erfordert eine genaue Schätzung der Restlaufzeit

- Kann zu häufigen Kontextwechseln führen
- Risiko: *Starvation* von größeren Prozessen möglich
- Verbleibende Bearbeitungsdauer erforderlich?: Ja
- Scheduling-Typ: präemptiv

3.5. Selektierung der Algorithmen

3.5.1. Ausschlusskriterien

Wenn ein Schedulingalgorithmus folgende Eigenschaften aufweist, dann wird dieser in dem späteren teil der Arbeit nicht mehr betrachtet, da Das bestehende System diese Anforderungen nicht her gibt:

- **Präemptiv?:** Wenn der Algorithmus präemptiv ist, also laufende Prozesse unterbricht um Prozesse mit höherer Priorität vorzuziehen, dann ist dieser Algorithmus nicht geeignet, da der Fortschritt nicht gespeichert werden kann und der abgebrochene Prozess dann von vorne starten muss
- **Muss die Bearbeitungszeit bekannt sein?:** Wenn die Bearbeitungszeit bekannt sein muss, um die Aufträge zu sortieren, dann kann dieser Algorithmus verworfen werden, da die Bearbeitungszeiten nicht bestimmt und auch nicht gut vorhergesagt werden können.
- **Müssen feste Prioritäten vorgegeben sein?:** Wenn feste Prioritäten vorgegeben sein müssen, dann kann der Algorithmus ebenso verworfen werden, da wir keine festen Prioritäten vergeben wollen

3.5.2. **Ausschluss ungeeigneter Algorithmen**

Tabelle 3.1 bewertet alle vorgestellten Algorithmen anhand dieser drei Kriterien. Ein **Ja** in einer der Spalten führt zum Ausschluss des Algorithmus.

Algorithmus	Präemptiv?	Bearbeitungszeit nötig?	Feste Prioritäten?	Ergebnis
Non-Preemptive Priority Scheduling	Nein	Nein	Ja	Ausgeschlossen
Preemptive Priority Scheduling	Ja	Nein	Ja	Ausgeschlossen
First In First Out (FIFO)	Nein	Nein	Nein	Verbleibt
Deadline Monotonic Scheduling (DMS)	Nein	Nein	Nein	Verbleibt
Least Slack Time First (LSF)	Ja	Ja	Nein	Ausgeschlossen
Highest Response Ratio Next (HRRN)	Nein	Ja	Nein	Ausgeschlossen
Shortest Job First (SJF)	Nein	Ja	Nein	Ausgeschlossen
Shortest Remaining Time First (SRTF)	Ja	Ja	Nein	Ausgeschlossen

Tabelle 3.1.: Bewertung der Scheduling-Algorithmen anhand der Ausschlusskriterien

3.6. Zusammenfassung

Die Anforderungen an den OptimScheduler umfassen:

- **Funktionale Anforderungen:** Der Scheduler muss Optimierungsaufträge parallel verarbeiten, eine Warteschlange verwalten, Ressourcen überwachen und Laufzeiten begrenzen können.
- **Technische Anforderungen:** Der Scheduler wird als Java-basierter REST-Webservice mit Spring Boot implementiert, mit JSON-Datenaustausch und Unterstützung für parallele Prozessausführung.
- **Scheduling-Strategien:** Verschiedene Algorithmen (FIFO, Priority-Based, LLR, Adaptive) wurden analysiert. FIFO und Priority-Based Scheduling werden als Implementierungsfokus empfohlen.
- **Architekturelle Flexibilität:** Die Implementierung soll modular aufgebaut werden, um verschiedene Strategien austauschbar zu ermöglichen und experimentelle Evaluierungen zu unterstützen.
- **Auswahl der Algorithmen:** Die Algorithmen First In First Out und Deadline Monotonic Scheduling passen als einzige zu den definierten Ausschlusskriterien und werden somit als einzige weiter betrachtet und analysiert.

Diese Anforderungen bilden die Grundlage für die Konzeptionierung des OptimSchedulers im folgenden Kapitel.

4. Konzeptionierung

4.1. Datenstrukturen

Die Aufträge für den Webservice werden mit definierten Datenstrukturen verwaltet. Die Ein- und Ausgabe-Datentypen sind wie folgt definiert.

4.1.1. Eingabe-Datentypen

- **Optimierer** (String): Auswahl des Optimierers (z.B. ftlopt, coflo, MIXOR)
- **Eingabedatei** (JSON/CSV-Objekt): Eingabedatei für den Optimierer im JSON/CSV-Format
- **maxLaufzeit** (Integer, optional): Maximale Laufzeit in Sekunden
- **Authentifizierung** (String): Authentifizierungstoken oder API-Key

4.1.2. Ausgabe-Datentypen

- **AuftragsID** (UUID): Eindeutige identifikationsnummer des Auftrags
- **Ergebnis** (JSON/CSV-Objekt): Die Ausgabedatei der Optimierung (bei erfolgreicher Verarbeitung)
- **Zeitstempel** (Objekt): Zeitpunkte: Eingang, Optimierungsstart, Beendet (ISO 8601)
- **Fehlermeldung** (String): Fehlerbeschreibung, wenn ein Fehler auftritt
- **Ressourcenverbrauch** (Objekt): stellt den Verbrauch der Hardware Ressourcen dar

4.2. Aufbau des Webservice

Dieser Abschnitt beschreibt den groben Ablauf und den grundlegenden Aufbau des OptimSchedulers als Webservice, einschließlich seiner Einbettung in die bereits vorhandene Infrastruktur.

4.2.1. Überblick

Der OptimScheduler ist ein selbstständiger Webservice, der als Docker-Container betrieben wird. Der Webservice kapselt die gesamte Logik für die Steuerung der Ausführung der Optimierungsprozesse. Dadurch entkoppelt er gleichzeitig diese vom Tserv. Es können Optimierungsprozesse direkt über eine REST-Schnittstelle und nicht nur über eine Dateischnittstelle vom Tserv aufgerufen werden.

Eine wesentliche Designentscheidung des Webservices ist, dass jeder Optimierungsprozess als eigener **Kubernetes-Pod** gestartet wird. Dadurch werden die einzelnen Optimierungsprozesse vom Host-System isoliert; Ressourcen können somit pro Pod gezielter festgelegt werden und dadurch ist auch eine flexible Skalierung über die Kubernetes-Infrastruktur möglich.

4.2.2. Ablauf

Der typische Ablauf einer Optimierungsanfrage ist wie folgt:

1. Ein externer Aufrufer (z. B. der Tserv) sendet einen POST /optimize-Request mit Optimiererauswahl, Eingabedaten und optionaler Laufzeitbegrenzung.
2. Der OptimScheduler authentifiziert die Anfrage und validiert die Eingabe.
3. Sind Ressourcen verfügbar und die maximale Parallelität noch nicht erreicht, startet der OptimScheduler über die Kubernetes-API einen neuen Pod. Andernfalls wird der Auftrag in die Job-Queue eingereiht und wartet, bis ein Slot frei wird.
4. Der gestartete Pod führt den Optimierer aus und schreibt das Ergebnis zurück.
5. Der OptimScheduler leitet das Ergebnis an den ursprünglichen Aufrufer weiter und gibt Laufzeitinformationen sowie Ressourcenverbrauch zurück.

4.2.3. Systemarchitektur

Abbildung 4.1 veranschaulicht den beschriebenen Aufbau. Der OptimScheduler bildet die Brücke zwischen dem externen Aufrufer und dem Kubernetes-Cluster, in dem die Optimierer-Pods ausgeführt werden.

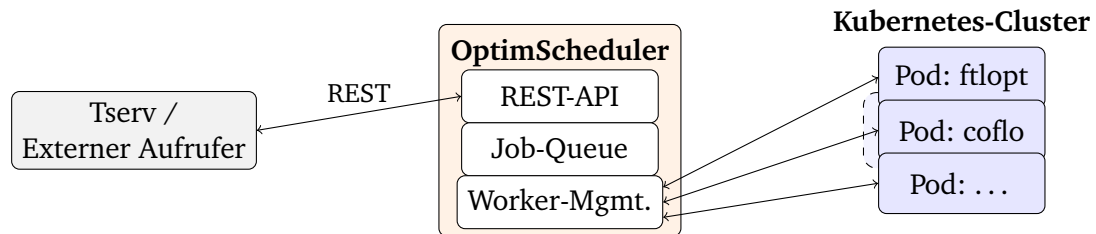


Abbildung 4.1.: Systemarchitektur des OptimSchedulers: Der Webservice nimmt Anfragen per REST entgegen, verwaltet eine Job-Queue und startet Optimierer als Pods in einem Kubernetes-Cluster.

4.2.4. Technologie-Stack

Der OptimScheduler wird als Java-Anwendung mit dem Framework Spring Boot implementiert und mit Maven gebaut. Tabelle 4.1 gibt einen Überblick über die eingesetzten Technologien.

Komponente	Technologie
Backend / Scheduler	Java, Spring Boot
Build-Tool	Maven
Containerisierung	Docker
Prozessorchestrierung	Kubernetes
Schnittstelle	REST / HTTPS / JSON
Betriebssystem	Linux

Tabelle 4.1.: Technologie-Stack des OptimSchedulers

4.3. Beschreibung der ausgewählten Algorithmen

Nun werden die zwei Algorithmen, die nach der Vorselektion verbleiben - FIFO und Deadline Monotonic Scheduling - detailliert beschrieben und mit ein paar Beispielen veranschaulicht.

4.3.1. First In First Out (FIFO)

Funktionsweise

FIFO ist ein relativ einfaches Scheduling-Verfahren: Die eingehenden Optimierungsaufträge werden in einer Warteschlange gesammelt und nach und nach in der Reihenfolge abgearbeitet, in der sie angekommen sind. Ein laufender Prozess wird dabei nicht unterbrochen (nicht präemptiv).

Der größte Vorteil von FIFO liegt in der Vorhersehbarkeit. Jeder Auftrag wird irgendwann ausgeführt. Dadurch wird *Starvation*, also das Übergehen eines Auftrags, vermieden. Zudem hängt die Wartezeit eines Prozesses ausschließlich von den zuvor eingegangenen Prozessen ab.

Ein bekanntes Problem des FIFO Algorithmus ist der sogenannte *Convoy Effect*. Das bedeutet, dass ein langwieriger Prozess die Warteschlange blockiert, wobei die Dringlichkeit nicht berücksichtigt wird. In einem Szenario, in dem zeitkritische Aufträge bearbeitet werden sollen, kann das zu unerwünschten Verzögerungen führen.

Schematische Darstellung

Abbildung 4.2 zeigt den grundlegenden Aufbau einer FIFO-Warteschlange. Neue Aufträge werden am hinteren Ende eingereiht und am vorderen Ende entnommen.

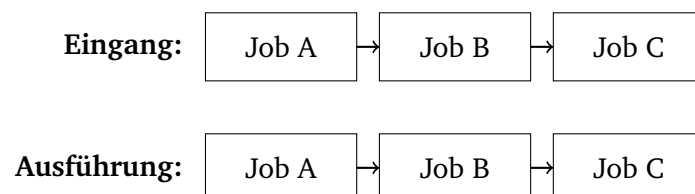


Abbildung 4.2.: Eingangs- und Ausführungsreihenfolge bei FIFO – Reihenfolge bleibt unverändert

Beispiel

Das folgende Beispiel zeigt, wie eine FIFO Warteschlange in einem System funktionieren kann. Wichtig zu beachten ist, dass in dem System die Dauer nicht bekannt ist. Zudem zeigt das Beispiel, wie ein *Convoy Effect* auftreten kann.

Auftrag	Dauer (min)	Deadline (min)	Eingangsreihenfolge
Job A	5	12	1
Job B	2	4	2
Job C	3	8	3

Tabelle 4.2.: Auftragsparameter für das Beispiel

In der Tabelle lässt sich erkennen, dass erst Job A, dann Job B und dann Job C ankommt. Da Job A Job B blockiert, obwohl Job B zeitkritischer ist verpasst Job B seine Deadline. Ebenso blockieren Job A und B Job C, sodass Job C die Deadline verpasst.

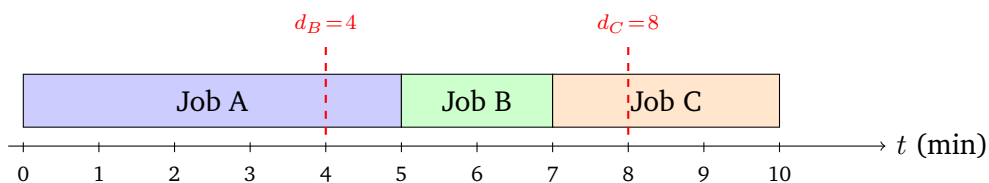


Abbildung 4.3.: Gantt-Diagramm FIFO: Deadlines von Job B und Job C werden verpasst

4.3.2. Deadline Monotonic Scheduling (DMS)

Funktionsweise

Beim Deadline Monotonic Scheduling ist die Ausführungsreihenfolge über die Deadlines der Aufträge bestimmt. Dabei hat der Auftrag mit der nächstliegenden Deadline die höchste Priorität. Diese Priorität wird ausschließlich aus der Deadline bestimmt und ist damit implizit. Es muss also keine Priorität von außen mitgegeben werden.

In Verbindung mit dem OptimScheduler lässt sich die `maxLaufzeit` aus dem Anfrage-Payload als Deadline ableiten. Die Deadline ergibt sich dann aus dem Eingangszeitpunkt und der maximalen Laufzeit ($Deadline = t_{Eingang} + \text{maxLaufzeit}$).

Wenn keine `maxLaufzeit` definiert ist, dann erhält der Auftrag die Deadline unendlich und hat damit automatisch die niedrigste Priorität.

Da DMS ein nicht präemptiver Algorithmus ist, wird kein bereits laufender Prozess unterbrochen. Neue Aufträge werden in die Warteschlange beim Eingang einsortiert. Zeitkritische Aufträge sollen durch die Priorisierung nach Deadlines zeitnah abgearbeitet werden.

Schematische Darstellung

Die Abbildung 4.4 zeigt, wie die Aufträge aus dem Beispiel in die DMS Warteschlange angeordnet werden. Die Sortierung erfolgt aufsteigend nach der Deadline.

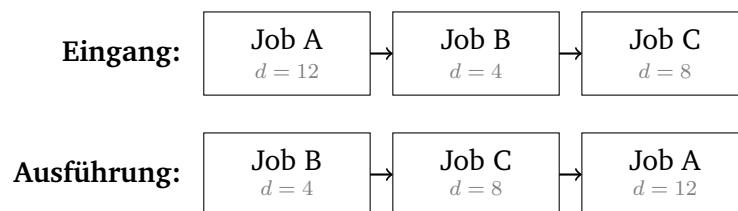


Abbildung 4.4.: Eingangs- und Ausführungsreihenfolge bei DMS – Umsortierung nach Deadline

Beispiel

Auftrag	Dauer (min)	Deadline (min)	Ausführungsreihenfolge (DMS)
Job B	2	4	1
Job C	3	8	2
Job A	5	12	3

Tabelle 4.3.: Ausführungsreihenfolge bei DMS – sortiert nach Deadline

Für dieselben drei Aufträge aus Tabelle 4.2, sowie Tabelle 4.3, ergibt sich bei DMS die Ausführungsreihenfolge: Job B (engste Deadline $d = 4$), dann Job C ($d = 8$), zuletzt Job A ($d = 12$). Abbildung 4.5 stellt diese Reihenfolge dar. Alle drei Deadlines werden eingehalten.

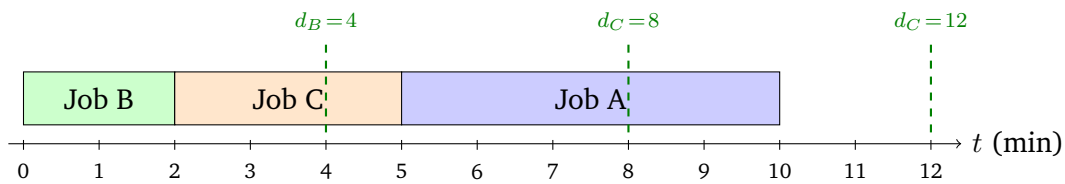


Abbildung 4.5.: Gantt-Diagramm DMS: Alle Deadlines werden eingehalten

In diesem Beispiel werden jetzt die Deadlines der einzelnen Jobs eingehalten. Dies ist allerdings nur sinnvoll, wenn es auch nicht durch Überlastung des Systems zu *Starvation* kommt.

4.3.3. Vergleich der Algorithmen

Tabelle 4.4 stellt die wesentlichen Eigenschaften beider Algorithmen gegenüber.

Kriterium	FIFO	DMS
Implementierungskomplexität	Gering	Mittel
Priorisierung	Keine	Implizit über Deadline
Starvation-Risiko	Nicht vorhanden	Sehr gering
Benötigte Zusatzinformation	Keine	maxLaufzeit
Eignung für zeitkritische Aufträge	Gering	Hoch
Convoy-Effect-Risiko	Vorhanden	Reduziert

Tabelle 4.4.: Eigenschaftsvergleich von FIFO und DMS

4.4. Analyse der Algorithmen

4.4.1. Implementierung der Prototypen

Beide Algorithmen, also DMS und FIFO, werden als Prototyp im Webservice austauschbar implementiert. Der Algorithmus kann dann über eine Konfigurationseinstellung gewechselt werden, damit diese miteinander verglichen werden können.

4.4.2. Messgrößen

Für die Analyse der Algorithmen werden folgende Kennzahlen ausgewertet:

- **Durchschnittliche Wartezeit:** Mittlere Zeitspanne zwischen Eingang eines Auftrags und dem Start seiner Ausführung
- **Durchschnittliche Durchlaufzeit:** Mittlere Zeitspanne zwischen Eingang und Abschluss eines Auftrags
- **Deadline-Einhaltungsrate:** Anteil der Aufträge, deren Deadline eingehalten wurde
- **Maximale Wartezeit:** Längste aufgetretene Wartezeit über alle Aufträge eines Szenarios
- **Systemdurchsatz:** Anzahl der abgeschlossenen Aufträge pro Zeiteinheit

4.4.3. Versuchsaufbau

Die Messungen werden anhand von vier Szenarien durchgeführt, die typische Betriebssituationen des OptimSchedulers abbilden:

- **Szenario 1 – Gleichmäßige Last:** Gleichmäßig eingehende Aufträge mit ähnlichen Laufzeiten und ohne explizite Deadlines; dient als Baseline-Vergleich
- **Szenario 2 – Gemischte Dringlichkeit:** Aufträge mit unterschiedlichen Deadlines, die eine typische Mischung aus RT-, OEV- und PP-Optimierungen simulieren
- **Szenario 3 – Spitzenlast:** Viele gleichzeitig eingehende Aufträge, um das Verhalten unter hoher Auslastung zu erfassen
- **Szenario 4 – Zeitkritische Aufträge:** Aufträge mit engen Deadlines, gemischt mit langlaufenden Aufträgen ohne Deadline

Die Ergebnisse der Messungen sowie deren Auswertung und Interpretation folgen im nächsten Kapitel.

5. Evaluation

6. Fazit & Ausblick

Eidesstattliche Erklärung

Ich versichere hiermit an Eides statt, dass ich die vorliegende Arbeit selbstständig und ohne Benutzung anderer als der angegebenen Hilfsmittel angefertigt habe. Alle Stellen, die wörtlich oder sinngemäß aus Veröffentlichungen entnommen wurden, sind als solche kenntlich gemacht.

Im Rahmen der Erstellung dieser Arbeit wurde das KI-System 'KI.connect.nrw' unterstützend zur sprachlichen Überarbeitung sowie zur fachlichen Reflexion und Präzisierung eigenständig entwickelter Argumente genutzt. Eine Übernahme von KI-generierten Texten oder inhaltlichen Lösungsvorschlägen erfolgte nicht. Sämtliche fachlichen Aussagen, Bewertungen und Schlussfolgerungen wurden eigenständig erarbeitet und verantwortet. Die Nutzung erfolgte im Einklang mit der Zweckbestimmung des Systems sowie unter Beachtung datenschutz- und urheberrechtlicher Vorgaben.

Ort, Datum: _____ Unterschrift: _____